

AOC2 2nd Course Project - Data Memory Hierarchy

Javier Resano, José Luis Briz, and Alejandro Valero

Dept. of Computing Science and Systems Engineering, EINA - Univ. Zaragoza

Submission deadline: May 12th 9:00 a.m.

README

- **Save time by carefully reading this guide.**
 - This guide effectively serves as an extended lecture on Computer Architecture and Organization, covering details that are difficult to fully address in the classroom.
 - It will help clarify key concepts that may still be unclear despite the theoretical material and problem sets.
 - It describes the steps you must follow, the specific actions you are expected to take, and provides guidance for organizing and writing your report.
 - **Do not change component or signal names** in the provided VHDL source code without consulting the faculty beforehand.
 - All materials (source files, slides, problem sets, guides, etc.) are subject to the Intellectual Property Rights of the University of Zaragoza. **Any violation of these rights may have legal consequences.**
 - Please report any typos or errors you may find, and feel free to ask questions when clarification is genuinely required.
 - Developing the ability to extract and apply technical information from written documentation is an essential part of this course.
 - You are therefore expected to read this guide carefully before seeking assistance.
 - *Documented behavior should be consulted before assuming undefined behavior.*
-

Contents

1 Summary	2
2 Baseline system	2
2.1 SoC	3
2.2 Memory bus	3
2.3 MC	4
2.4 MD Scratch	4
2.5 MC Controller	4
2.6 MD Controller	5
2.7 Event Counters in MC	5
2.8 IO_Master	5
3 Results and Deliverables	5
3.1 Tests	5
3.1.1 Unit Tests	6
3.1.2 Integration Tests	6
3.2 Project Report	6
4 Grading Policy	7
5 Optional Extensions	7

1 Summary

Building on the first project, you will design a cache memory controller for a more complex memory hierarchy. Components communicate over a semi-synchronous bus and comprise a cache memory (MC), a data memory (MD), and a scratch data memory (MD Scratch). The MD runs at lower speed than the MD Scratch, whose contents are *non-cacheable*. The MC includes performance counters. Presentation and defense follow the same procedures as before. Read this guide, its annexes, and the submission instructions carefully.

2 Baseline system

The memory system comprises a cache memory (MC), a data memory (MD), a *scratch* memory (MD Scratch), a semi-synchronous bus, and an I/O peripheral (IO_Master). Both the MC and IO_Master controllers act as bus managers¹, so the bus requires an arbiter with dynamic priority assignment to prevent conflicts. The MIPS interface is identical to that of the first project. Fig. 1 shows the bus signal diagram.

¹Following the 2021 IEEE SA Standards Style Manual on inclusive language, we avoid master/slave in favor of descriptive terms.

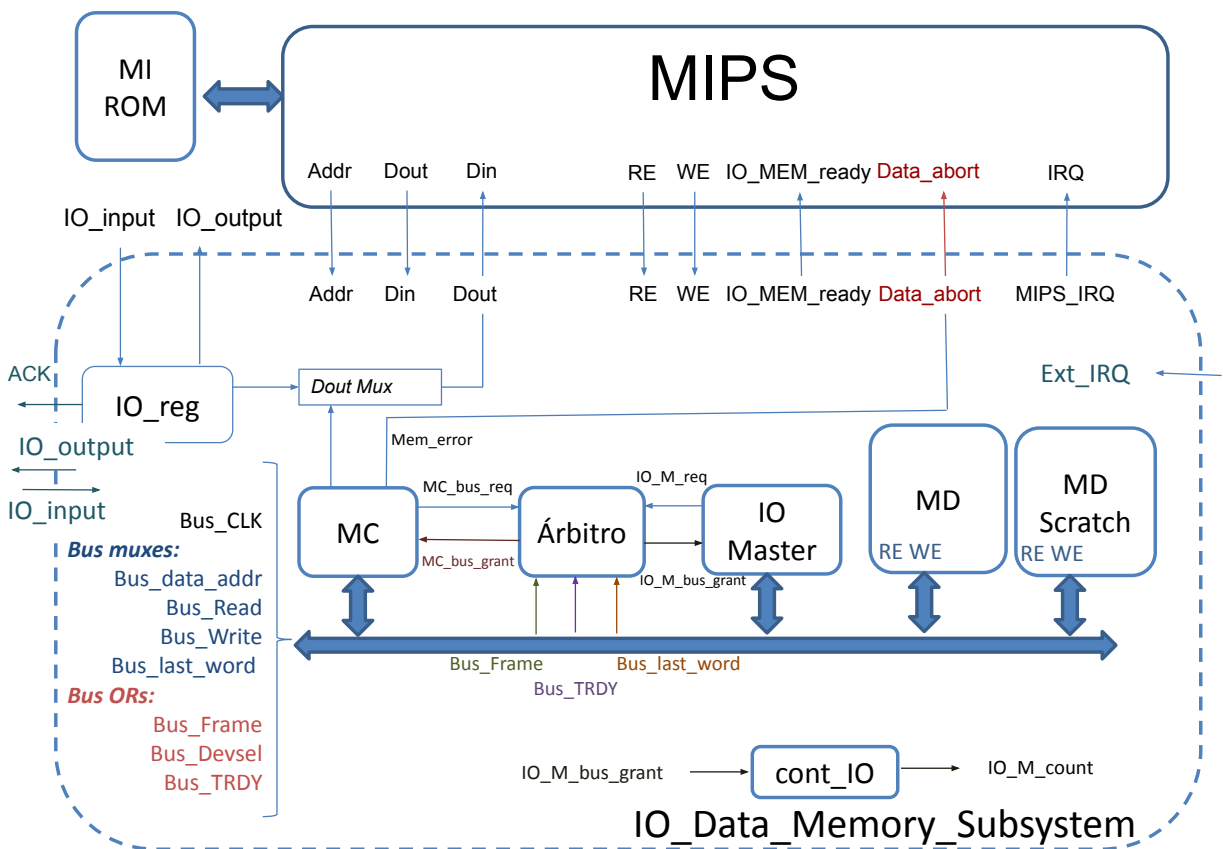


Figure 1: Principal components and signals in the IO_MD_subsystem. The assignment consists in designing the MC Control Unit.

2.1 SoC

Reuse all first-project SoC and MIPS modules unchanged, along with the five components you submitted. Replace `IO_MD_subsystem.vhd` with the new version and add its auxiliary components. The signal `IO_MEM_ready` is asserted the cycle `IO_Data_Memory_Subsystem` completes a RE/WE request from the MIPS. Unlike the previous project, such requests may now take several cycles.

2.2 Memory bus

This semi-synchronous bus is loosely based on the PCI standard. MC and IO_Master act as managers; MD and MD_Scratch are always clients. **Address and data lines are multiplexed.** The bus supports variable-length bursts across three phases:

- **Arbitration** – A manager requests the bus via `Bus_req`; the arbiter asserts `Bus_grant` when the bus is free. Since there are two managers, waits are possible.
- **Address** – A transfer starts the cycle after `Bus_grant` is asserted. The manager asserts `Bus_Frame`, drives the address onto `Bus_data_addr`, and sets `Bus_Rd_Wr` ('0' = read, '1' = write). **The client whose address space matches asserts `Bus_DevSel` in that same cycle.**

- **Caution:** If no device asserts `Bus_DevSel`: a) the address is stored in `ADDR_Error_Reg`; b) the signal `MEM_Error` is set; and c) The MIPS' UC triggers a `Data_abort`. The register is readable at address `x"01000000"`; reading it clears `MEM_Error`.
- Two additional sources trigger `MEM_Error`: unaligned accesses (as in the first project), and any write attempt to `ADDR_Error_Reg`, which is read-only.
- **Data** – Begins **the cycle after `Bus_DevSel` is asserted**. The manager holds `Bus_Frame` throughout. Each cycle, the target asserts `Bus_TRDY` when ready and transfers one word over `Bus_data_addr`; if it cannot meet timing, it deasserts `Bus_TRDY` to insert a wait. Words are always transferred consecutively (e.g., words 4, 8, 12...). The manager deasserts `Bus_Frame` after the last word, at which point the target removes its data from the bus; **`last_word` is asserted alongside the final transfer** (and also for a single-word transaction).

2.3 MC

The MC is a two-way ($s=2$) set-associative cache with 128 bytes (32 words), FIFO replacement, copy-back write policy, and write-around on write miss. Each way holds 4 blocks of 4 words.

2.4 MD Scratch

The MD Scratch is a small, fast memory with the same access latency as the MC, managed explicitly via `lw/sw` within `x"10000000"–x"100000FF"`. Accesses to this range cause no faults and involve no data exchange with MD. Addresses in this range are *non-cacheable*: the MC controller forwards any MIPS read or write targeting MD Scratch directly onto the bus, **without fetching a block** into MC. On `lw`, the returned word is forwarded to the MIPS. Burst mode is not supported; words are accessed one at a time.²

2.5 MC Controller

The MC controller handles requests from the MIPS, responding with data from MC or I/O registers when possible, or performing the required transfers with MD/MD Scratch via the bus. A full-page schematic of MC's internal organization is provided in the Moodle project materials as a design reference. **The controller is a FSM** (Finite State Machine) of moderate complexity; the main challenge is to fully understand the protocols and their signals.

The MC controller also includes a **secondary FSM** with two states (`Memory_error` and `No_error`), which must be implemented either within the main FSM process or separately. This FSM drives the `Mem_Error` signal, interpreted by the MIPS as a `Data_abort`. The transition from `No_error` to `Memory_error` occurs when:

- a) no device acknowledges the sent address;
- b) the MIPS requests a misaligned word; or
- c) the MIPS attempts to write to a read-only register.

In the error state, the controller continues normal operation while keeping `Mem_Error` asserted, and **stores the address (or byte) of the faulting word in the `Addr_Error_Reg` register**. The controller **remains in the error state until the MIPS reads `Addr_Error_Reg`**, memory-mapped at address `x"01000000"`, at which point it returns to `No_error`.

²Early GPGPUs lacked caches but included this type of *scratch* memory — called *shared memory* in NVIDIA/CUDA and *scratchpad* in OpenCL. Since the Fermi architecture, NVIDIA has offered a level configurable as cache, scratchpad, or a mix. Scratchpads are also common in embedded SoCs. Our MD Scratch is inspired by this concept.

2.6 MD Controller

The MD controller always acts as a target, forwarding bus requests to MD for addresses within its space (`0x00000000`–`0x000001FF`) and ignoring all others.

Timing: this is the same MD from the first project, with added delays: the latency for the first word (L) exceeds that for subsequent words (R)³. When MD cannot supply data in the current cycle, it deasserts `Bus_TRDY` to signal that the controller must wait.

Burst mode: when `Bus_Frame` is asserted, the data memory executes the requested operation at the initial address and then increments it internally.

2.7 Event Counters in MC

The MC includes **four counters**⁴:

1. **cont_m:** misses in MC — accesses (lw or sw) to a **cacheable address** not present in MC. Accesses to MD Scratch or MC internal registers are not counted.
2. **cont_w:** writes to MC. Writes to I/O registers or MD Scratch are not counted.
3. **cont_r:** reads from MC. Reads from I/O registers or MD Scratch are not counted.
4. **cont_cb:** cache blocks written to MD during dirty-block replacement.

Caution: each event must increment its counter exactly once. For example, if handling a miss takes 23 cycles, `cont_m` increments once, not 23 times. Track reads, writes, misses, and replacements in your test-bench and verify they match the MC counter values visible in the waveform — useful both for debugging and performance evaluation.

2.8 IO_Master

The system includes an input/output peripheral (`IO_Master`) acting as a bus manager. It monitors `IO_input` every cycle and continuously attempts to write its value into the first word of MD Scratch. The counter `cont_IO` increments each cycle in which `IO_Master` successfully acquires the bus. The more efficiently the MC controller manages transfers, the more bus cycles are available to `IO_Master` — and the higher `cont_IO` will be.

3 Results and Deliverables

3.1 Tests

Verify that your design handles all relevant cases correctly: read/write hits, read/write misses across all sets and ways (both clean and dirty), and correct counter increments throughout. Also perform reads and writes to MD Scratch, trigger a memory error, confirm that the `abort` signal is asserted, and verify it clears upon reading the internal MC register. To this end, we recommend the following testing methodology.

³Recall the baseline bus model explained in class.

⁴Real controllers often include more counters to evaluate performance and guide program adaptation to the memory hierarchy.

3.1.1 Unit Tests

Use a testbench to isolate and test specific scenarios. A provided *testbench* example covers read misses, hits, and clean replacements across the entire MC — run it on your design and incrementally add further scenarios: write hits and misses, dirty replacements, reads and writes to MD Scratch, I/O registers, internal MC registers, abort handling, and so on. A no-IRQ *testbench* is also included to simplify debugging.

3.1.2 Integration Tests

Write an assembly program covering the cases tested in the testbench. A single comprehensive program can expose side effects that shorter, isolated tests may miss. Document and comment this code in your project report, as in the first project.

3.2 Project Report

Begin with a short summary (100 words) of your design. Avoid lengthy prose; use code/pseudocode snippets, figures, and diagrams throughout. The following items are mandatory:

1. **Control Unit state graph**, with the operations and active signals per state. A per-state and/or per-transition table can add detail while keeping the graph readable. Focus on which signals change and why, especially the key ones.
2. **Address breakdown** figure for MC addressing, as covered in class ([tag | set | word | byte/word]).
3. **Bus transfer latency analysis**, excluding arbitration delays. Obtain the following from simulation:
 - $CrB(MD)$: cycles to read a block from MD = $L + 3 \times R$ cycles.
 - $CwW(MD)$: cycles to write a word to MD.
 - $CrW(MDscratch)$: cycles to read a word from MD Scratch.
 - $CwW(MDscratch)$: cycles to write a word to MD Scratch.
 - $CrW/CwW(IO_REG)$: cycles to read from or write to internal I/O registers.
4. **Average cycles per memory access**: mathematical formulation as a function of the costs above and of all possible events (read/write hit; read/write miss to MD or MD Scratch; access to internal MC registers). Verify uncertain values via simulation. Note that **arbitration latency is not constant**: assume an average of 1.5 cycles.
5. **Unit tests**: testbench or program snippets, commented, targeting specific events — e.g., a read miss followed by a stream of hits, misses on the same set triggering replacement, a read miss followed by a write hit with and without replacement, accesses to MD Scratch, etc. Cover whatever cases best demonstrate correctness. Identify all FSM states and **provide a table specifying where and how each case was verified**.
6. **Integration test**: assembly program exercising the memory hierarchy. Compute the **speedup** of MC+MD Scratch over an equivalent system with only the MD of this project (not from the 1st Project Assignment). Reuse a testbench program if appropriate. Speedup may be computed theoretically or from cycle counts obtained via simulation (with and without cache).
7. **Time log**: hours each group member dedicated to each part, and the **total hours**.
8. **Acknowledgements**: first and last names of anyone who assisted in the development of the project.
9. **Individual self-assessment**, answering:
 - Do you consider the course objectives met?
 - What grade would you give yourself?

4 Grading Policy

Each submission is assessed individually using our own test program. As in the 1st Project Assignment, **a correct design is required to pass**. Minor errors will be flagged with an annotated testbench and pre-loaded memory files; you will have 24 hours to fix and re-submit.

The state graph must be clear and easy to follow. Rather than crowding the diagram with all signals, use a table to detail per-state (Moore) or per-transition (Mealy) behavior, referencing states and transitions by number or label. **Your tests must specify whether each `lw` or `sw` results in a hit or miss, and indicate set and way for cached data. An absent or incorrect analysis means we will not evaluate your design.**

Signal assignments — particularly those of the MC control unit — must be clearly justified, in both the report and the interview. Answers such as “*if I put ‘0’ instead of ‘1’ it doesn’t work*” are not acceptable. This is not meant to intimidate: 99% of you will have no issues. We simply require that submissions reflect genuine understanding. This applies equally to re-submissions, which are reviewed thoroughly; even minor bugs may prompt a deeper inspection, and any inconsistencies found will result in a failing grade.

Grading criteria: the primary criterion is whether the **state diagram is correct, clear, and well explained**, and whether the simulation is consistent with your description and average memory access formula. Secondary criteria include controller efficiency and stall minimization. Start with a straightforward design and optimize later if time permits (see options below).

5 Optional Extensions

- *Basic lockup-free cache.* Include a **write buffer to MD**: when a write to MD is required, the data and address are stored in dedicated registers and the processor is released by asserting `Mem_Ready`. The MC may continue serving hits while the Control Unit handles the bus write. Execution stalls only if a second bus transfer is required before the first completes. Identify cases where this mechanism yields a measurable performance gain.
- *Ethical hacking.* Implement and evaluate a function that degrades performance by flooding MC with dirty blocks, forcing an eviction on every miss. Assess its detectability in the SoC and discuss possible countermeasures.
- *Critical word forwarding.* On a read miss, the MC controller normally fetches the full block before forwarding the requested word to the CPU. This extension forwards the requested word as soon as it arrives, allowing the CPU to resume while the remainder of the block is retrieved. Describe scenarios where this technique offers a meaningful performance improvement.